

Linear Classifiers and the Linear SVM

D.A. Forsyth, UIUC

Linear Classifiers

- Key ideas
 - data $(\mathbf{x}_i, y_i)$
 - $\mathbf{x}_i$ are feature vectors, dimension d
 - y_i are labels, and y_i is either 1 or -1
 - query is $\mathbf{x}$
 - predicted label is:

$$\text{sign}(\mathbf{a}^T \mathbf{x} + b)$$

Bias, variance, irreducible error

- Assume we choose a classifier from some family
 - eg, a particular linear classifier from the family of linear classifiers
- The error on future data decomposes into three terms:

Bias + Variance + Irreducible error



The error caused by the fact that even the best classifier in the family makes errors



The error caused by the fact that we didn't get the best classifier in the family, just nearly



Unavoidable error (recall alien example)

Question: how to choose a and b ?

- Rough answer:
 - split dataset into train/test
 - choose a and b to get good behavior on train
 - how? details?
 - evaluate on test

Question: how to choose a and b?

- Choose a and b to get good behavior on train
 - write a cost function $C(a, b)$ and find best
- The cost function:
 - generally, each training example should contribute evenly
 - average of per example costs

$$C(\mathbf{a}^T, b) = \left(\frac{1}{N}\right) \sum_{i=1}^N c(\mathbf{a}, b, \mathbf{x}_i)$$

Desirable properties $c(a, b, \mathbf{x}_i)$

- Write:

$$\gamma_i = \mathbf{a}^T \mathbf{x}_i + b$$

- sign is the predicted label
- magnitude is prop to distance to line

- We want:

- Prediction $\neq$ true label:
 - $c(a, b, \mathbf{x}_i)$ should be big. Should get bigger when magnitude is bigger, but not too fast
- Prediction == label, mag small:
 - $c(a, b, \mathbf{x}_i)$ should be non-zero, getting smaller as magnitude gets bigger
- Prediction==label, mag big:
 - $c(a, b, \mathbf{x}_i)$ should be zero

The hinge loss

$$c(\mathbf{a}, b, \mathbf{x}_i) = \max(1 - (\mathbf{a}^T \mathbf{x}_i + b) y_i, 0) = \max(1 - \gamma_i y_i, 0)$$

Feature vector for example

True label for example

$$\gamma_i = \mathbf{a}^T \mathbf{x}_i + b$$

The hinge loss

$$c(\mathbf{a}, b, \mathbf{x}_i) = \max(1 - (\mathbf{a}^T \mathbf{x}_i + b) y_i, 0) = \max(1 - \gamma_i y_i, 0)$$

- Notice:
 - Prediction $\neq$ true label:
 - $c(\mathbf{a}, b, \mathbf{x}_i)$ is big. Gets bigger when magnitude grows, but not too fast
 - Prediction $=$ label, mag small:
 - $c(\mathbf{a}, b, \mathbf{x}_i)$ is non-zero, getting smaller as magnitude gets bigger
 - Prediction $=$ label, mag big:
 - $c(\mathbf{a}, b, \mathbf{x}_i)$ is zero

Regularization

- The cost function drives us towards a classifier that does well on training data
 - but what about future data?
- Cases:
 - future data item is correctly classified, large mag
 - nothing to do
 - future data item is correctly classified, small mag
 - OR is incorrectly classified
 - Notice:
 - hinge loss on this data item (which we don't know) can be scaled
 - and this doesn't change the classification of the training data

if $(\mathbf{a}^T \mathbf{x} + b)y < 1$ and $s > 0$, we have $c(s\mathbf{a}, sb, \mathbf{x}_i) = (1 - s(\mathbf{a}^T \mathbf{x}_i + b)) y_i$

Regularization

- IDEA:
 - Hinge loss on future data items can be reduced by scaling a, b
 - Equiv:
 - find “small” a, b that produce low cost on training
 - Equiv:
 - penalize large a during training
- Which gets us

$$S(\mathbf{a}, b; \lambda) = \left[(1/N) \sum_{i=1}^N \max(0, 1 - y_i (\mathbf{a}^T \mathbf{x}_i + b)) \right] + \lambda \left(\frac{\mathbf{a}^T \mathbf{a}}{2} \right).$$

Regularization weight

Data term; empirical risk

Regularization term

Choosing a classifier

- Minimize this expression with respect $\mathbf{a}$, b

$$S(\mathbf{a}, b; \lambda) = \left[(1/N) \sum_{i=1}^N \max(0, 1 - y_i (\mathbf{a}^T \mathbf{x}_i + b)) \right] + \lambda \left(\frac{\mathbf{a}^T \mathbf{a}}{2} \right).$$


Data term; empirical risk

Regularization term

Choosing a classifier, II

- Write $\mathbf{u} = \begin{bmatrix} \mathbf{a} \\ b \end{bmatrix}$ for cost function $g(\mathbf{u})$
- Usual procedure:
 - start at $\mathbf{u}^{(0)}$
 - repeat:
 - find a descent direction $\mathbf{p}^{(i)}$
 - choose a steplength η
 - update $\mathbf{u}^{(i+1)} = \mathbf{u}^{(i)} + \eta \mathbf{p}^{(i)}$

Choosing a classifier, III

- What is descent direction?
 - gradient descent:
 - Newton's method:
 - second derivatives help choose
- What is steplength?
 - small number
 - search for a good one

$$-\nabla_{\mathbf{u}}g = - \begin{bmatrix} \frac{\partial g}{\partial u_1} \\ \dots \end{bmatrix}$$

$$S(\mathbf{a}, b; \lambda) = \left[(1/N) \sum_{i=1}^N \max(0, 1 - y_i (\mathbf{a}^T \mathbf{x}_i + b)) \right] + \lambda \left(\frac{\mathbf{a}^T \mathbf{a}}{2} \right).$$

Stochastic gradient descent

- Assume we choose k data items uniformly at random
 - compute

$$C_k = \frac{1}{k} \sum_{i=1}^k c(\mathbf{a}, b, \mathbf{x}_i)$$

- We have

$$\mathbb{E}[C_k] = \frac{1}{N} \sum_{i=1}^N c(\mathbf{a}, b, \mathbf{x}_i)$$

What is the gradient?

Stochastic gradient descent, II

- IDEA:

- repeat:

- pick one or few data items uniformly at random
 - compute gradient using those
 - it isn't right, but
 - expected value is right
 - it's quick
 - take step backward down this gradient



How far? it's too hard to search because it's too hard to evaluate $g(u)$ so fixed, but may change over time (eg. 1000 steps at $1e-3$; 1000 steps at $1e-4$; etc).

Convergence: too hard to test; instead,
do this a fixed large number of steps, and monitor error rate

Stochastic gradient descent, III

- IDEA:
 - repeat:
 - pick one or few data items uniformly at random
 - compute gradient using those
 - it isn't right, but
 - expected value is right
 - it's quick
 - take step backward down this gradient

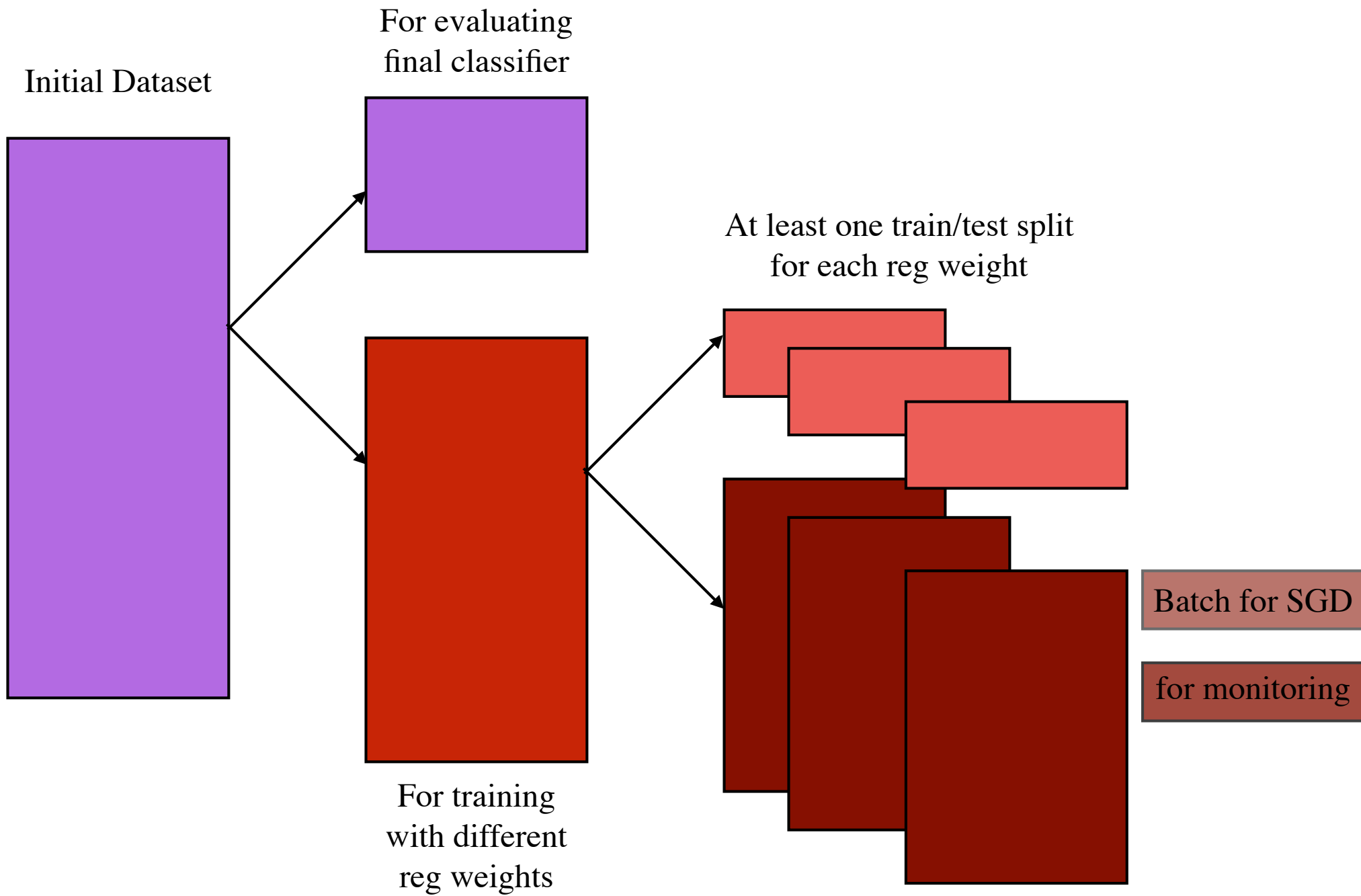
Convergence: too hard to test;
instead, do this a fixed large
number of steps, and monitor error rate

General points

- Support vector machine (SVM):
 - A linear classifier trained with hinge loss
- SVM's:
 - Impressively reliable if you have good features
 - Easy and effective
 - Good evidence you don't need to see all the training data
 - ie SGD might get you to about the right classifier without seeing all training data

Getting the regularization weight

- Strategy:
 - try various values, choose the one that yields best classifier
 - not super sensitive - search by factors of 10
- Remember:
 - we can't evaluate a classifier on data used to train it!
 - this makes training and evaluating an SVM slightly elaborate
- Constraints:
 - we need to know how well the final classifier works - split off some data
 - for each value of reg. weight, we must evaluate - split off some data
 - we may need to monitor - split off some data



Behavior

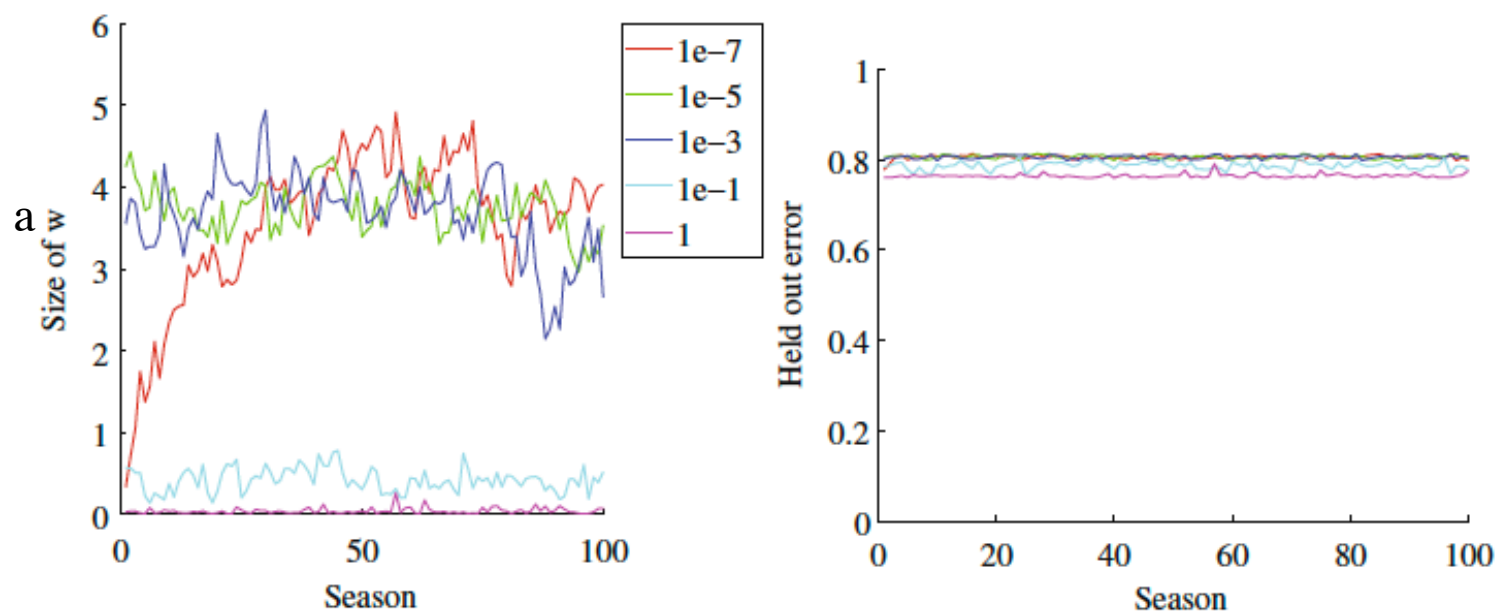


Fig. 11.3 On the *left*, the magnitude of the weight vector $\mathbf{a}$ at the end of each season for the first training regime described in the text. On the *right*, the accuracy on held out data at the end of each season. Notice how different choices of regularization parameter lead to different magnitudes of $\mathbf{a}$; how the method isn't particularly sensitive to choice of regularization parameter (they change by factors of 100); how the accuracy settles down fairly quickly; and how overlarge values of the regularization parameter do lead to a loss of accuracy

Notice we have a recipe...

- We could apply to other cost functions

$$c(\mathbf{a}, b, \mathbf{x}_i) = \log \left(1 + e^{-[(\mathbf{a}^T \mathbf{x}_i + b)y_i]} \right)$$

- this is logistic regression
- We could apply SGD to other functions
 - neural networks, etc.
- Procedure for selecting reg. weight is general
 - other cases

